

How to Think Like a Programmer

A Complete Guide to Logic
Building and Problem-
Solving

By Nidhish | The Entangle

PART 1: FOUNDATIONS

CHAPTER 1: What Programming Really Is

The Big Misconception

Most people think programming is about writing code in C++, Python, or Java. This is wrong.

Programming is about solving problems. Code is just the way you express your solution.

The 90-10 Rule

Here is the reality:

90 percent of programming is:

- Understanding what problem you are solving
- Thinking through how to solve it
- Breaking it into clear steps
- Planning your approach
- Designing the logic

10 percent of programming is:

- Writing the actual code
- Translating your logic into C++ syntax

Think of it this way: if you spend 2 hours on a programming problem, you should spend about 1 hour and 48 minutes thinking and planning, and only 12 minutes actually typing code.

A Simple Example

Problem: Add two numbers and show the result.

What most beginners do:

They immediately open the editor and start typing:

```
int a, b;  
cin >> a >> b;  
cout << a + b;
```

They get confused. They make mistakes. They do not understand why it works or does not work.

What good programmers do:

First, they think (2 minutes):

Step 1: What do I need to do?

- Get two numbers from the user
- Add them together
- Show the result

Step 2: What information do I need to store?

- First number
- Second number
- The sum

Step 3: In what order?

- First, I need to create places to store the numbers

- Then, get the first number from the user
- Then, get the second number
- Then, add them together
- Finally, show the answer

Then, they code (1 minute):

```
int num1, num2, sum;  
cin >> num1 >> num2;  
sum = num1 + num2;  
cout << sum;
```

Same result, but they understand exactly what each line does and why it is there.

Why Code is Just Translation

Think of programming languages like human languages.

The idea (universal): "Check if a person can vote"

In English: If the person's age is 18 or more, they can vote. Otherwise, they cannot vote.

In Hindi: Agar umar 18 ya usse zyada hai, to vote kar sakte hain. Nahi to nahi kar sakte.

In C++:

```
if (age >= 18) {  
    cout << "Can vote";  
} else {  
    cout << "Cannot vote";  
}
```

Same logic. Different words. That is all code is - translating your thinking into a language the computer understands.

The Real Skill

Companies do not hire you because you know C++ syntax.

They hire you because you can think clearly and solve problems.

Why? Because:

- Syntax you can look up on Google in 2 seconds
- Problem-solving ability takes practice and skill
- Languages change, but thinking skills do not

Master the thinking, and any programming language becomes easy to learn.

Key Takeaway

- Programming = 90% thinking + 10% typing code
- Learn to think clearly first.
- The code will follow naturally.

CHAPTER 2: Two Different Brains

Your Brain (Human)

Let me give you a problem: Find the biggest number in this list: 5, 2, 9, 1, 7

What did you do? You probably looked at the numbers and in about one second thought "9".

But here is the question: How did you know?

You probably cannot explain it. Your brain just saw the numbers and recognized that 9 is the biggest. It happened automatically. You did not consciously think through steps.

This is how human brains work - we process information quickly, recognize patterns, and make connections without being aware of all the steps involved.

Computer Brain (Machine)

Now imagine explaining to someone who has never seen numbers before how to find the biggest number. You would have to say:

1. Look at the first number (5)
2. Say to yourself "5 is the biggest number I have seen so far"
3. Look at the second number (2)
4. Ask yourself "Is 2 bigger than 5?"
5. The answer is no
6. So 5 is still the biggest
7. Look at the third number (9)
8. Ask yourself "Is 9 bigger than 5?"
9. The answer is yes
10. So now 9 is the biggest
11. Look at the fourth number (1)
12. Ask yourself "Is 1 bigger than 9?"
13. The answer is no
14. So 9 is still the biggest
15. Look at the fifth number (7)
16. Ask yourself "Is 7 bigger than 9?"
17. The answer is no
18. So 9 is still the biggest

19. There are no more numbers

20. The answer is 9

This is how computers work. They need every single step spelled out clearly. They cannot make assumptions or skip steps.

The Key Differences

You:

- Think quickly
- Skip steps automatically
- Use common sense
- Handle vague instructions
- Process multiple things at once
- Make assumptions based on context

Computer:

- Needs every step explained
- Cannot skip anything
- Has no common sense
- Needs exact, clear instructions
- Does one thing at a time
- Makes zero assumptions

Real-World Example

You tell a friend: "Make me tea"

What happens: Your friend automatically knows:

- Where the kitchen is
- What tea is
- How much to make
- That you probably want sugar
- To bring it to you when done

Perfect tea arrives.

You tell a computer: "Make me tea"

What happens:

ERROR: "tea" is not defined

ERROR: "make" is not a valid command

ERROR: "me" is not recognized

PROGRAM CRASHED

To make a computer do it, you would need to say:

1. Go to the room at coordinates (5, 10)
2. This room is called "kitchen"
3. Find the object called "kettle"
4. Pick up the kettle using 5 Newtons of force
5. Move to the sink
6. Turn the handle marked "cold"
7. Fill the kettle with 250 milliliters of water
8. Turn off the handle
9. Place kettle on the stove

10. Turn the stove knob to position "high"
... and about 40 more detailed steps ...
11. Carry the cup to the person who requested it

Now it works. But notice how much detail was needed.

Why This Matters for You

When you program, you are doing something unusual for a human brain. You are translating from:

How you naturally think → How computers need to be told

Your automatic understanding → Step-by-step instructions

Your quick recognition → Detailed, explicit commands

For your whole life, you have been thinking quickly and automatically. Now you need to think slowly and explicitly. This feels strange at first.

This is why programming feels hard when you start. Not because you are not smart enough. Not because you are bad at it. But because you are learning a completely new way of thinking.

The good news? This is a skill you can learn with practice.

The Bridge

Good programmers use both types of thinking:

Step 1: Use human thinking

- Understand the problem creatively
- Recognize patterns
- Come up with solutions

Step 2: Use computer thinking

- Break the solution into detailed steps
- Make everything explicit
- Remove all assumptions

Step 3: Write the code

- Translate those explicit steps into C++ syntax

Key Takeaway

You naturally think quickly and automatically.

Computers need slow, step-by-step instructions.

Programming is learning to think in steps while solving problems creatively.

This feels unnatural at first.

That is normal. Everyone struggles with this.

But it is completely learnable through practice.

CHAPTER 3: What is Logic Building?

Simple Definition

Logic building is the skill of breaking down problems into clear, step-by-step instructions that anyone (including a computer) can follow.

Real-Life Example

Problem: Cross the road safely.

Without logic building: "Just cross when it is safe"

This is too vague. What does "safe" mean? When exactly should you cross?

With logic building:

1. Walk to the edge of the road
2. Stop before stepping onto the road
3. Look to your left to check for vehicles
4. Look to your right to check for vehicles
5. Look to your left again
6. If you see any vehicles coming, wait
7. If you do not see any vehicles, walk straight across
8. Keep walking until you reach the other side
9. Step onto the sidewalk
10. You have successfully crossed

This is detailed. This is explicit. Anyone following these exact steps will cross safely.

This detailed way of thinking is what we call logic building.

The 5 Core Skills

Every programmer uses these five thinking skills to solve problems:

1. **Sequential Thinking** - Understanding that the order of steps matters.
Some things must happen before other things.
2. **Conditional Thinking** - Making decisions based on conditions. "If this is true, do this. Otherwise, do that."

3. **Iterative Thinking** - Handling repetition. When you need to do the same thing multiple times.
4. **Decomposition** - Breaking big, scary problems into smaller, manageable pieces.
5. **Pattern Recognition** - Noticing when different problems can be solved using similar approaches.

Master these five skills, and you can solve any programming problem.

Why It Matters More Than Syntax

Let me show you two scenarios:

Scenario 1: You know syntax but not logic

Problem: Find if a number is prime.

You know:

- How to write if statements
- How to write for loops
- How to print output

But you do not know:

- What makes a number prime
- How to check if a number divides evenly
- What approach to use

Result: You cannot solve the problem. The syntax knowledge does not help.

Scenario 2: You know logic but not syntax

Problem: Find if a number is prime.

Your logic:

- A prime number can only be divided by 1 and itself
- So I need to check if any other number divides it
- I will check every number from 2 to (number - 1)
- If any of them divides evenly, it is not prime
- If none of them divide evenly, it is prime

You do not know:

- The exact C++ syntax for loops
- The exact symbols to use

Result:

- You understand the problem completely
- You know exactly how to solve it
- You can explain the solution
- You can look up the C++ syntax on Google in 2 minutes
- Problem solved

The lesson:

Good logic + No syntax = Easy to solve (just look up syntax)

No logic + Good syntax = Impossible to solve (stuck completely)

This is why we focus on logic building first.

What Logic Building Looks Like in Practice

Problem: Calculate the average of three numbers.

Your logic building process:

Step 1: What information do I need?

- Three numbers (this is input)
- Their average (this is output)

Step 2: How do I calculate average?

- Add all three numbers together
- Divide the sum by 3 (because there are 3 numbers)

Step 3: In what order should I do things?

1. Get the first number
2. Get the second number
3. Get the third number
4. Add them: $\text{sum} = \text{first} + \text{second} + \text{third}$
5. Divide: $\text{average} = \text{sum} \text{ divided by } 3$
6. Show the average

Step 4: Any special considerations?

- What if the numbers have decimal points? (I should use float instead of int)
- What if dividing gives a decimal? (float handles this automatically)

The logic is now complete.

Only now do you start writing code to express this logic in C++.

Notice: You solved the entire problem in your head before touching the keyboard.

Key Takeaway

- Logic building is the core skill.
- Syntax is just the tool to express your logic.
- Strong logic + Any language = Success
- Weak logic + Best language = Failure
- Focus on building strong logic first.
- The syntax will come easily after that.